

Job Market Analytics Dashboard

Step 3 — Data Cleaning with Pandas

Step 3 Overview

Status: ✓ Complete

Created scraper/clean_data.py which takes the raw JSON from the Adzuna API, normalizes all fields into a consistent structure using Pandas, and extracts skill tags from job titles and descriptions using keyword matching. Successfully cleaned 20 live job postings with all fields populated.

3.1 Why Data Cleaning is Needed

Raw API data is messy — nested dicts, missing fields, inconsistent formats. Before we can store or analyze it, we need to flatten and normalize it. Pandas is the industry-standard Python library for this.

Raw API Problem	Clean Data Solution
company is nested: {"display_name": "XACT..."}	Flatten to: company = "XACT PLACEMENTS LIMITED"
location is nested: {"display_name": "London, UK"}	Flatten to: location = "London, UK"
salary_min / salary_max can be None	Keep as None — handle in charts later
No skills field in API response	Extract from title + description using keyword matching
description is hundreds of words	Truncate to 500 chars for storage efficiency
created is ISO timestamp string	Keep as-is, convert to datetime when needed

3.2 Install Pandas

```
pip install pandas
pip freeze > requirements.txt
```

requirements.txt now includes pandas and its dependencies (numpy, python-dateutil, pytz).

3.3 scraper/clean_data.py — Full Code

```
import pandas as pd

# Master list of skills we scan for in job titles and descriptions
SKILLS_LIST = [
    "python", "flask", "django", "fastapi",
    "sql", "postgresql", "mysql", "mongodb",
    "pandas", "numpy", "scikit-learn", "tensorflow", "pytorch",
    "machine learning", "deep learning", "nlp",
    "javascript", "react", "node.js", "typescript",
    "docker", "kubernetes", "aws", "azure", "gcp",
    "git", "linux", "rest api", "graphql",
    "data analysis", "data science", "tableau", "power bi",
]

def extract_skills(text):
    """Scan text for known skills, return list of matches."""
    if not text:
        return []
    text = text.lower()
    return [skill for skill in SKILLS_LIST if skill in text]

def clean_jobs(raw_jobs):
    """Normalize raw API job list into a clean Pandas DataFrame."""
    cleaned = []
    for job in raw_jobs:
        title = job.get("title", "").strip()
        company = job.get("company", {}).get("display_name", "").strip()
```

```

location = job.get("location", {}).get("display_name", "").strip()
salary_min = job.get("salary_min")
salary_max = job.get("salary_max")
description = job.get("description", "")
source_url = job.get("redirect_url", "")
posted_date = job.get("created", "")
category = job.get("category", {}).get("label", "")
# Extract skills from title + description combined
skills = extract_skills(title + " " + description)
cleaned.append({
    "title": title,
    "company": company,
    "location": location,
    "salary_min": salary_min,
    "salary_max": salary_max,
    "skills": skills,
    "category": category,
    "source_url": source_url,
    "posted_date": posted_date,
    "description": description[:500],
})
df = pd.DataFrame(cleaned)
return df

if __name__ == "__main__":
    from fetch_jobs import fetch_jobs
    raw = fetch_jobs()
    df = clean_jobs(raw)
    print(df[["title", "company", "location", "salary_min", "skills"]].to_string())

```

3.4 How Each Part Works

Code	What it does
<code>job.get("company", {}).get("display_name", "")</code>	Safely extracts nested field — returns "" if missing
<code>description[:500]</code>	Truncates description to 500 chars to save database space
<code>extract_skills(title + " " + description)</code>	Combines title and description then scans for skill keywords
<code>[skill for skill in SKILLS_LIST if skill in text]</code>	List comprehension — returns only skills found in the text
<code>pd.DataFrame(cleaned)</code>	Converts list of dicts into a Pandas DataFrame (table)
<code>.to_string()</code>	Prints the full DataFrame without truncating columns

3.5 What is a Pandas DataFrame?

A DataFrame is like an Excel spreadsheet in Python — rows and columns, where each row is one job and each column is a field (title, company, salary, etc.). It makes filtering, grouping, sorting, and aggregating data very easy. This is the same structure we will later use to generate chart data.

3.6 Actual Output (20 jobs cleaned)

#	Title	Company	Location	Salary Min	Skills Extracted
0	Backend Python Developer	XACT PLACEMENTS LIMITED	London, UK	60,000	[python, azure]
1	Python Developer - mthree	mthree	London, UK	66,663	[python]
2	Senior Django/Python Developer	Sanderson	London, UK	156,000	[python, django]
3	Junior Python Developer (Hedge Fund)	FinancialCareers	London, UK	43,400	[python]
4	Data Infrastructure Engineer	Corriculo Ltd	London, UK	80,000	[python]

11	Python Developer	GCS	Farringdon, Central London	60,000	[python, sql, aws, azure, gcp, power bi]
18	Python Developer	NEEV LIMITED	London, UK	58,169	[python, flask, django, fastapi, sql, react]
19	Python Developers	CGI	London, UK	83,405	[python]

Result: ✓ 20 jobs cleaned. Skills extracted correctly from descriptions.

3.7 SKILLS_LIST — Why Keyword Matching?

The Adzuna API does not return a structured skills field. Skills are buried inside the job description text. Our approach: maintain a master list of known tech skills and scan every description for matches. Simple, fast, and extensible — just add more skills to SKILLS_LIST anytime.

Category	Skills in SKILLS_LIST
Languages	python, javascript, typescript, sql
Frameworks	flask, django, fastapi, react, node.js, graphql
Databases	postgresql, mysql, mongodb
ML / Data	pandas, numpy, scikit-learn, tensorflow, pytorch, machine learning, deep learning, nlp
Cloud	aws, azure, gcp
Tools	docker, kubernetes, git, linux, tableau, power bi

Build Progress

Step	Description	Status
1	Project setup — venv, packages, file structure, local Flask server	✓ Done
2	Adzuna API — register, get keys, write fetch_jobs.py, pull live data	✓ Done
3	Data cleaning — Pandas, normalize fields, extract skill tags	✓ Done
4	Supabase — create jobs table, store first batch	Next →
5	Dashboard route — query Supabase, pass data to templates	
6	Plotly charts — skills, location, salary charts	
7	Jobs listing page — /jobs route with filters	
8	UI polish — dark CSS theme	
9	GitHub + Railway deployment	
10	Add to portfolio via /admin	
11	(Stretch) ML salary prediction model	